Einrichtung & Erste Bildverarbeitung

Digitale Bildverarbeitung

Gruppenarbeit

- Gruppen a 2-3 Personen
- Programmierübungen in Python
- Abgaben:
  - Python-Files
  - Bilder: Input/Output

- Bewertung
  - Bewertung der Übungsaufgaben (35 %)
    - 1. Aufgabe – 5 %
    - 2 & 3. Aufgabe – je 15 %
    - Bonuspunkte durch Zusatzaufgaben (insgesamt 10% möglich)
  - Klausur in Moodle oder als abzugebende Dokumente (65 %)

Gruppenarbeit

Übung 1

- Ziel:
  - Python - Entwicklungsumgebung einrichten
  - Grundlegende Kenntnisse im:
    - Laden, Anzeigen & Speichern von Bildern
    - Zugriff auf Pixel und Channels
    - Verändern von Pixeln

Programmierumgebung | Python

- Betriebssystem-unabhängige, interpretierte Programmiersprache
- Einfache Einrichtung → Niedrige Einstiegshürde
- Erweiterbar durch Packages (z.B. OpenCV, Numpy etc.)
- Aktuelle Version: 3.13 (Stand 01.2025)
- Im Kurs ab Version 3.8 verwendbar
- Wichtige Packages: Numpy, SciPy, Matplotlib, Opencv, PyQt6

Anaconda

- Open-Source-Distribution für Python
- Bietet “Virtual Environments” für verschiedene Projekttypen
- Package-Management
- https://www.anaconda.com/products/individual

Entwicklungsumgebung | PyCharm

- IDE von JetBrains
- Kostenlose **Community** Version
- **Professional** Version „For Educational Purpose“ kostenlos
  → Anmeldung mit DHBW-Mail nötig
- Anpassung von Python-Environments innerhalb der IDE möglich
- Autovervollständigung, Debugging, Variable-Watch
- Alternative IDEs:
  - Spyder, Visual Studio Code, Sublime Text, Eclipse / Pydev
  - Editor + Python-Umgebung in der Konsole
- https://www.jetbrains.com/de-de/pycharm
- OpenCV-ImageViewer-Plugin:

  https://plugins.jetbrains.com/plugin/14371-opencv-image-viewer/versions

Zusammenfassung

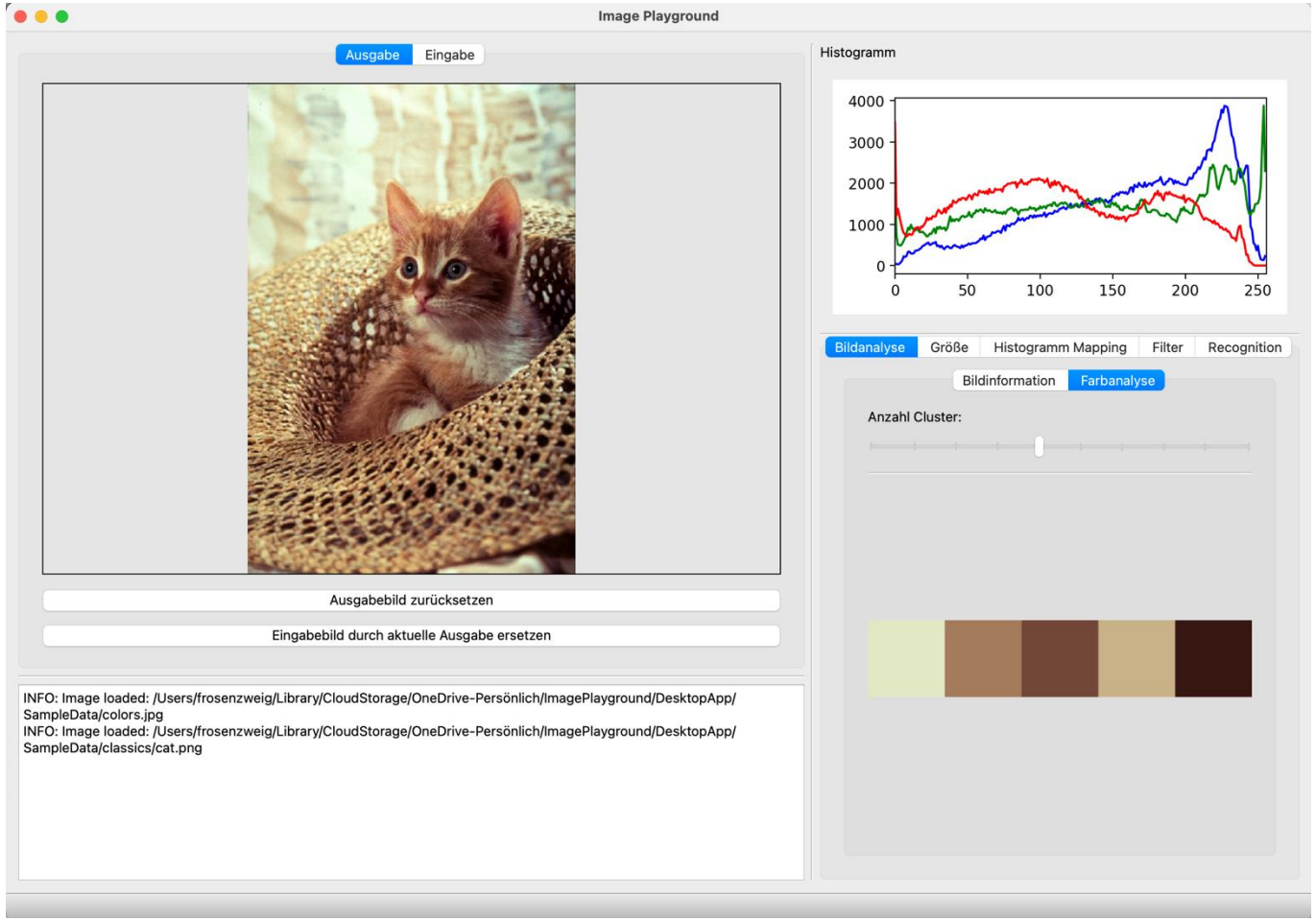
- Beispiele in Python 3.8

- Package-Management über Anaconda
  - Wichtige Packages:
    Numpy, SciPy, Matplotlib, Opencv, PyQt6
  - https://www.anaconda.com/products/individual

- Entwicklungsumgebung - PyCharm:
  - https://www.jetbrains.com/de-de/pycharm
  - ImageViewer-Plugin

- Alternative IDEs:
  - Spyder, Visual Studio Code, Sublime Text, Eclipse / Pydev

ExerciseApp



- app.py
- AWSRekognition.py
- ColorAnalysis.py
- controllers.py
- HistogramManipulation.py
- ImageFiltering.py
- ImageInformation.py
- ImagePlotter.py
- loggers.py
- models.py
- MplWidget.py
- Playground_UI.py
- Playground_UI.ui
- Utilities.py
- views.py

Übungen

ExerciseApp - Requirements

- numpy
- matplotlib
- opencv-python
- PyQt6
- scikit-learn
- Boto3

- Hinweise für Anpassungen an der GUI:
  - GUI erstellt mit Qt Designer
  - pyuic6 Playground_UI.ui -o Playground_UI.py

Übung 1 | Aufgabe

- Code in der Datei ImageInformation.py

1. Bild laden & Bild anzeigen
2. Bild beliebig aber sichtbar verändern
3. Verändertes Bild anzeigen & speichern
4. Funktionen für Bildinformationen implementieren
5. Einzelne Channels eines Farbbildes separieren und abspeichern
6. Histogramm des Bildes anzeigen & speichern (vorgegebene Funktion nutzen!)

Abgabe: ImageInformation.py, Bilder und Histogramm

Hinweise

Bilder in OpenCV

- Bilder werden durch numpy-Arrays repräsentiert
- img = np.zeros((height,width,3), np.uint8)

For-Loop:

```
for i in range(start, end, step):
    for j in range(repetitions):
        doSomething(i, j)
```

Input/Output

- Read Image (0 - Grauwerte/ 1- Farbe):

```
img = cv2.imread(„path“, 0 / 1)
```

- Write Image:

```
cv2.imwrite(„path“, img)
```

- Display Image:

```
cv2.imshow( “Window Title”, img)
cv2.waitKey(0)
```

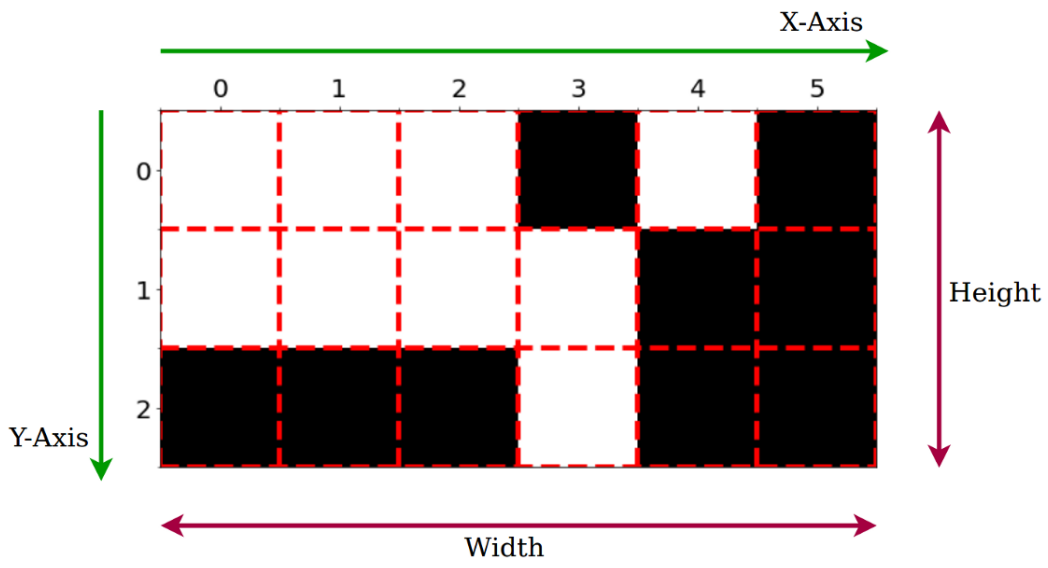
Hinweise | Bildzugriffe

Zugriff in Arrays:

- Image[Zeile, Spalte, Kanal] = Image[y , x , channel]
- Image[: , x , 2] → Zugriff auf alle Pixel des B Kanals in Spalte x
- Image[: , : , 0] → Zugriff auf alle Pixel des R Kanals
- Image[:, 10:20, :] = np.zeros([img.shape[0], 10, 3])
  → Setzt 3 Kanäle in den Spalten 10-20 auf 0

Schnellerer Zugriff in numpy-Arrays:

- Image.item(Zeile, Spalte)
- Image.itemset(Zeile, Spalte, Wert)



Übung 1

Viel Erfolg!